

Disclaimer: Dieses OER-Dokument stammt aus dem Projekt [proTirone](#), das über GitHub [freie Unterrichtsmaterialien](#) samt [Quellcode](#) offeriert. proTirone-Materialien werden unter den Bedingungen der [CC-BY-4.0-Lizenz](#) so angeboten, wie sie sind, ohne Zusage bestimmter Eigenschaften und ohne Gewährleistung (§5). Dafür dürfen sie – bei angemessener Namensnennung (§3) – für beliebige (auch kommerzielle) Zwecke verändert und weitergegeben werden (§2).

LF11c:03:Datenaufbereitung

[1] Dimensionen der Datenanalyse [→ ZP:Sheet:2]

Zweck von Daten: Sie sollen ...

- Weltbeschreibungen liefern.
- zur Weltverbesserung anregen.

Aber:

- Elektronische RAW-Daten sind nichts als Bits = Bytes = Zahlen
- Sie sagen für sich nichts.
- Sie müssen **syntaktisch** und **semantisch** interpretiert = **aufbereitet** werden.

[2] Begriffe / Definitionen

- **Interne Daten** :-
 - entstehen bei der Ausführung / Durchführung firmeneigener Prozesse
 - mit firmeneigenen Systemen (ggfls. auch angemietet (Cloud))
 - mit Komponenten in firmeneigenen Netzen
- **Externe Daten** :-
 - entstehen durch Fremdfirmenprozesse in firmenfremden Netzen
 - können frei angeboten werden (offizielle Statistiken)
 - können frei = legitim ausgelesen werden (Web-Crawler)
 - können von Fremdanbietern gekauft werden
- **Datenqualität** :-
 - meint die Güte der Daten, mit der sie einen Weltausschnitt abbilden und inwieweit sie weiterverwendet werden können:
 - wird nach gesonderten Kriterien begutachtet
 - **Vollständigkeit** :-

- * Erfassen die Daten den Weltausschnitt hinreichend umfassend?
- * Sind sie für die Weiterverarbeitung hinreichend komplett?

– **Korrektheit** :-

- * Erfassen die Daten den Weltausschnitt hinreichend genau?
- * Ist die Datenerfassung transparent?
- * Sind Manipulationen ausgeschlossen?
- * Wie wahrscheinlich sind Ablesefehler?

Anmerkung: Die Kriterien *Vollständigkeit* und *Korrektheit* stammen aus der KI/Mathematik:

- Ein Deduktionsalgorithmus ist vollständig, wenn er alle natürlichen Schlussfolgerung auch aus sich heraus ableitbar macht.
- Ein Deduktionsalgorithmus ist korrekt, wenn all seine aus sich heraus ableitbaren Schlüsse auch natürlich legitime Schlussfolgerungen sind.
- Ein vollständiger und korrekter Deduktionsalgorithmus ist z.B. der, den das Wissensrepräsentationssystem PROLOG verwendet.

Beispiel:

- Ein Algorithmus zur Auflistung der Schüler der Klasse 12ip/iv23 ist **korrekt**, wenn er nur Schülerinnen der Klasse 12ip/iv23 ausgibt.
- Ein Algorithmus zur Auflistung der Schüler der Klasse 12ip/iv23 ist **vollständig**, wenn er alle Schülerinnen der Klasse 12ip/iv23 ausgibt.
- Wäre er nur *vollständig*, könnte er - außer alle Schülerinnen der Klasse 12ip/iv23 - auch noch die der Klasse 12ia23 ausgeben.
- Wäre er nur *korrekt*, könnte er einige Schülerinnen der Klasse 12ip/iv23 und keine anderen ausgeben, müsste aber nicht alle auflisten.
- **Konsistenz** :- meint Widerspruchsfreiheit
 - *innere Konsistenz* :- die innere Widerspruchsfreiheit der Daten (= die Daten dürfen nicht aussagen, dass 'etwas etwas in derselben Hinsicht zugleich zukommt und nicht zukommt') [Aristoteles]
 - *äußere Konsistenz* :- die Daten stehen nicht mit anders gewonnenen Daten im Widerspruch.

Beispiel:

Es gibt aktuell zwei Verfahren, die Gravitationskonstante zu messen. Die Verfahren wären für sich unbrauchbar, wenn sie in sich inkonsistente Messdaten lieferten. Die Daten der Verfahren widersprechen allerdings denen der anderen. Dies führt zwar zu einem Konflikt. Dessen Existenz allein desavouiert aber nicht die Datenqualität.

- **Redundanzfreiheit**

- meint, dass dieselbe Tatsache in den Daten nicht mehrfach abgebildet/erfasst ist.
- Problem im Hinblick auf Verarbeitungsaufwand
- oft gefährdet bei der Zusammenführung (Mergen) von Daten

- **Verfügbarkeit**

- Sind die Daten technisch verarbeitbar?
- Sind die Daten rechtlich nutzbar?

- **Einheitlichkeit**

- Ist das syntaktische Format einer Datenmenge gleich?
- Stimmt das syntaktische Format einer Datenmenge mit dem gleicher, externer Daten überein?
= Standardkonform?

- **Datenverarbeitungsgrad**

- **strukturierte Daten** haben festes, zu Bedeutung passendes Format und können auf/mit einer der Datenerfassungsformate repräsentiert werden, nämlich in/mit
 - * JSON Dateien :- <https://de.wikipedia.org/wiki/JSON>
 - * XML Dateien :- https://de.wikipedia.org/wiki/Extensible_Marku_Language
 - * CSV Dateien :- [https://de.wikipedia.org/wiki/CSV_\(Dateiformat\)](https://de.wikipedia.org/wiki/CSV_(Dateiformat))
 - * INI-Dateien (Schlüssel-Wert-Paare) :- https://en.wikipedia.org/wiki/INI_file
 - * YAML-Dateien :- <https://de.wikipedia.org/wiki/YAML>
 - * → [protico.lessons: lf.cx.Datafiles](https://protico.lessons.org/lf/cx/Datafiles))
- **semi-strukturierte Daten** haben Format, das die Bedeutung nicht vollständig repräsentiert [Bilder etc.]
- **unstrukturierte Daten** = Sammlungen von Daten mit unterschiedlichen Formaten und unterschiedlicher Bedeutung.

Vorwegnahme:

In sehr großen Datenmengen machen obige Begriffe keinen Sinn mehr, weil das, worauf sie zielen, nicht mehr überprüfbar oder prozessual / technisch generierbar ist. Trotzdem sollen auch aus solchen Quellen Aussagen gewonnen werden. Dazu gibt es den **Cross Industry Standard Process for Data Mining** (= CRISP-DM):

- die Datengewinnung = das Geschäft verstehen
- Auswahl und Analyse der Daten, Bewertung im Hinblick auf Adäquatheit des zu erfassenden Geschäftsausschnittes
- (syntaktische) Datenaufbereitung

- Auswahl von Data-Mining-Methoden, Erstellung der Modellierung
- Evaluierung (exemplarischer Vergleich mit Weltausschnitten)
- Bereitstellung der Ergebnisse für eine Geschäftsverbesserung

[3] Sprung in die konkrete Datenanalyse [→ ZP:Sheet:3]

ÜBUNG LF11C:Datenanalyse:01

- ☐ Laden Sie sich die Datei `dsp.alpha.xyz` herunter.
 - ☐ Untersuchen Sie die darin enthaltenen Daten - notfalls mit einem HEX-Editor Ihrer Wahl.
 - ☐ Stellen Sie eine Hypothese darüber auf, worum es bei diesen Daten geht.
-

Dieser Lösung nähern wir uns schrittweise:

[4] Phasen der Datenanalyse [→ ZP:Sheet:4]

Analyse von Daten folgt immer dem Schema:

0. **Datensicherung:**

1. Nicht auf den Originaldaten arbeiten, Datensicherung anlegen.
2. Wenn Datenmenge dafür zu groß. Datenportionen abspalten.

3. **Syntaktische Datenaufbereitung:**

1. *Datenanalyse auf Byteebene (SYDA . A)*: Ermitteln Sie, was sich tatsächlich auf der “Platte” (**HEX-Editor**)
2. *Dateninterpretation auf Byteebene (SYDA . I)*: Ermitteln Sie, welche Bytes zusammengehören und welche Datentypen bilden.
3. *Datensatzformatanalyse auf Byteebene (SYDA . E)*: Ermitteln Sie die Datensätze und deren typische innere Struktur.
4. *Syntaktische Datenreparatur (SYDA . R)*: Gleichen Sie strukturell abweichende Datensätze wo sinnvoll syntaktisch oder löschen Sie die massiv dubiosen
5. *Datenkonverter (SYDA . K)*: Programmieren Sie einen Konverter mit wiederverwendbarer Zwischenrepräsentation.

4. **Semantische Auswertung:**

1. *Datenzugriff (Laden)* (SEDA . L): Recyclen Sie Ihren Konverter so, dass Sie programmintern Daten messen, zählen bzw. generell auswerten können.
2. *Semantische Datenkorrektur* (SEDA . K): Plausibilisieren Sie die Datensätze, korrigieren Sie sie wo möglich und löschen Sie 'Irrläufer'.
3. *Datenevaluation* (SEDA . E): Programmieren Sie die typischen Evaluationsfunktionen (Durchschnitt, Mittelwert, ...).
4. *Aggregation* (SEDA . A): Aggregieren Sie die ermittelten Trends, Aussagen etc. in einem geeigneten Format.
5. *Visualisierung* (SEDA . V): Entwickeln Sie ein präsentables Format für die aggregierten Aussagen.
6. *Präsentation* (SEDA . P): Präsentieren Sie die ableitbaren Tendenzen, Verhalten, Aussagen präsentabel.

Anmerkungen:

- Ohne syntaktisch bereinigte Daten muss die semantische Aufbereitung scheitern.
- Ohne Konverter (digital bearbeitbare Daten), können keine Trends berechnet werden.
- Ohne semantische bereinigte Daten werden Aussagen zu Trends etc. verfälscht.

Regeln:

1. Gehen Sie davon aus, dass Ihre Daten 'verrauscht' sind - egal, was man Ihnen verspricht.
2. Verlassen Sie sich nicht auf die Zusage, sie würden weitere Daten immer in exakt demselben Format bekommen. Niemand kann Ihnen das zusichern. Daten sind mit Tools generiert. Und die ändern sich.
3. Integrieren Sie die syntaktische Datenbereinigung deshalb nicht in den Konverter. Setzen Sie für beides eigene Entwicklungen ab. Nutzen Sie gerne auch unterschiedliche Tools
4. SYDA.A - SYDA.E mit Methoden des Shellscriptings.
5. SYDA.K mit fertigen Konvertern, Datenbibliotheken und Interpretersprachen
6. Schreiben Sie Ihren Konverter mit einer Zwischenrepräsentation, die Sie für spätere Mess- und Zählfunktionen wieder verwenden können.

[5] Probleme der Byte-Interpretation (SYDA.I) [→ ZP:Sheet:5]

- Bei der Datenanalyse der erste Blick immer mit einem HEX-Editor!
- Erst der zweite mit einem 'normalen' Code-Editor.

Warum: Editoren interpretieren Bytewerte und -folgen schon als Zeichen. Man sieht so nicht, welche Zahlen (Bytes) wirklich auf der Platte stehen.

Zur Frage der (Text)Codierung:

Ein Byte repräsentiert **entweder** eine Zahl oder ein Zeichen! Was davon in welchem Kontext ist Interpretationssache.

- Byte = Zahl: (s. https://en.wikipedia.org/wiki/C_data_types)
- `sizeof(int)` : früher 2 Bytes (Word), heute meist 32 Bits = 4 Bytes
- `sizeof(long)` : heute meist 64 Bits = 8 Bytes

Byte = Zeichen ergibt

- 7Bit-Ascii = (American Standard Code for Information Interchange) -
 - Zeichensatz: 7bit-Encodierung für den angelsächsischen Raum.
 - Wertebereich: 0x00 - 0x7f:
 - * 0x00–0x1F: nicht druckbare (Drucker) Steuerzeichen
 - * 0x20–0x2F: Satzzeichen, Sondersymbole
 - * 0x30–0x3F: Ziffern und mathematische Sondersymbole
 - * 0x40–0x5F: 26 Großbuchstaben, Klammern, (Satz)Zeichen
 - * 0x60–0x7F: 26 Kleinbuchstaben, Klammern, (Satz)Zeichen
- 8Bit-Ascii = ANSI (American National Standards Institute)
 - Zeichensatz: 8bit-Encodierung für den europäischen Raum.
 - Wertebereich:
 - * 0x00 - 0xFF:
 - 0x00–0x7F wie ASCII
 - 0x80–0xFF verschiedene Zeichencodebelegungen wie *iso-latin-1*, ..., *iso-latin-15*, *Windows-1252*

UNICODE = Universal Code

- ordnet allen weltweit genutzten Schriftzeichen zu eindeutigen Wert zu
- erkennbar an PräfixU+
- gefolgt von 4 bis 6 HEX-Zahlen zwischen 0000 und 10FFFF
- Beispiel U+00C4 = 'Ä'
- => bis jetzt gibt es 17 Ebenen zu je $2^{16} = 65536$ Zeichen = 154.998 Zeichen in der Version Unicode 16.0
- U+0000 – U+007F = entspricht 7Bit-ASCII Zeichenbelegung.
- Ein Zeichensatz, der alle UNICODE Zeichen drucken könnte, wäre ein Universal Coded Character Set

- Als Speichercode unpraktisch, weil viele Bytes mit 0en verschwendet.
- vgl. dazu [<https://de.wikipedia.org/wiki/Unicode>](<https://de.wikipedia.org/wiki/Unicode> “Unicode Systematik”)

Problem: Würde man nur Unicode verwenden, wären alle Texte sehr,sehr groß.

Lösung: UTF8 = Unicode Transformation Format - 8 Bit [**→ ZP:Sheet:6**]

- Jede Unicode-Zahl wird in eine Zahl bestehend aus 1 - 4 Bytes transformiert.
- Besteht die UTF-8-Zahl aus 1 Byte, ist das höchste Bit 0. Die restlichen Bits sind wie im 7Bit-Ascii mit Zeichen belegt.
- Besteht die UTF-8-Zahl aus 2 Bytes, beginnt das höherwertige Byte mit 2 gesetzten Bits, gefolgt von einem ungesetzten (110), das niederwertige Byte beginnt mit einem gesetzten und einem ungesetzten Bit (10)
- Besteht die UTF-8-Zahl aus 3 Bytes, beginnt das höherwertige Byte mit 3 gesetzten Bits und einem ungesetzten (1110), die beiden nachfolgenden Bytes beginnen jeweils mit einem gesetzten und einem ungesetzten Bit (10)
- Besteht die UTF-8-Zahl aus 4 Bytes, beginnt das höherwertige Byte mit 4 gesetzten Bits und einem ungesetzten (11110), die drei nachfolgenden Bytes beginnen jeweils mit einem gesetzten und einem ungesetzten Bit (10)

vgl. <https://de.wikipedia.org/wiki/UTF-8>

- UTF16 = Code-Transformation.
- Zweck beider ist es, häufige genutzte Zeichen mit möglichst wenig Bytes zu repräsentieren.

Die Abbildung der UTF-8-Zahlen auf die UNICODE-Zahlen legt das UTF-8-Symbol fest.

vgl. https://wiki.selfhtml.org/wiki/Zeichencodierung/Bytes_und_Buchstaben

Beispiel Ä

- = U+00C4 in Unicode
- = 0xC4 in UTF8
- = 196 in ISO-Latin-1
- = #196; als HTML-Zahl
- = Ä als HTML-Symbol

vgl. * <https://www.ascii-code.com/ISO-8859-1> * https://en.wikipedia.org/wiki/List_of_Unicode_characters * <https://www.fileformat.info/info/charset/UTF-8/list.htm> * <https://www.ascii-code.com/> * <https://www.w3schools.com/charsets>

[6] Verfahren beim Konverterbau [→ ZP:Sheet:7]**Ein Konverter**

- liest Daten aus einer Datei (bzw. von Stdin) ein,
- speichert sie typischerweise in einer Zwischenrepräsentation und
- schreibt sie im Zielformat in eine Datei (bzw. nach Stdout).
- kann die Daten datensatzweise einlesen, in einer ZRP-Form erfassen und jeden Datensatz sofort konvertieren.
 - *Vorteil:* auf große Datenmengen anwendbar
 - *Nachteil:* kontextsensitive Modifikationen i.d.R. nicht anwendbar
- kann alle Datensätze der Daten zusammen in einer ZRP-Form erfassen und danach iterativ konvertieren.
 - *Vorteil:* kontextsensitive Modifikationen gut anwendbar
 - *Nachteil:* Daten müssen insgesamt (oder in sehr großen Happen) ‘in-memory’ gehalten werden.
- verwendet typischerweise eine Adapterarchitektur:
 - Jedes Inputformat hat seinen eigenen Input-Adapter.
 - Verschiedene Input-Adapter lesen verschiedene syntaktische Formate in dieselbe ZRP ein.
 - Jedes Outputformat bekommt seinen spezifischen Output-Adapter.

Konverterarchitektur:

Das Konzept der ‘Zwischenrepräsentation’ (= Intermediate Representation)

- stammt aus dem Compilerbau (→ https://en.wikipedia.org/wiki/Intermediate_representation)
- kann immer verwendet werden, wenn Daten / Programme aus dem einen Format (‘Sourcecode’, ‘CSV’) in andere Formate (‘Binärcode für Architektur X, Y, Z’, ‘JSON, INI’) überführt (konvertiert) werden sollen.

Eine explizite Zwischenrepräsentation (mit hard-gecodeten Beispieldaten) mit Adapterarchitektur zu verwenden, bringt drei Vorteile:

1. Nach Definition und ‘Füllung’ der ZPR können Input- und Output-Adapter getrennt voneinander entwickelt werden.
2. Weitere Adapter für neue Formate können bestehende Vorarbeiten nutzen
3. Das Konverterprogramm bleibt leserlich.

Konverterentwicklung:

1. Von den zu konvertierenden Daten 10% der Datensätze als Testdaten abspalten.
2. Von den Testdatensätzen zwei möglichst komplexe als 'hart-zu-kodierende' ZPR-Initialisierung auswählen
3. Für die **Konverterprogrammierung** eine **MVP**- (= *Minimal Viable Product*)-**Strategie** nutzen:

Ein MVP, "[...] wörtlich (übersetzt:) ein 'minimal brauchbares oder existenzfähiges Produkt', ist die erste minimal funktionsfähige Iteration eines Produkts, die dazu dient, möglichst schnell aus Nutzerfeedback zu lernen und so Fehlentwicklungen an den Anforderungen der Nutzer vorbei zu verhindern."

(vgl. https://de.wikipedia.org/wiki/Minimum_Viable_Product)

ÜBUNG LF11C:Datenanalyse:02

Was wäre das MVP für einen Konverter für gegebene Inputdaten im Format IF und gewünschten Outputformat OF?

- ☐ Skizzieren Sie Ihre Möglichkeiten.
- ☐ Wählen Sie Ihre bevorzugte Variante und begründen Sie ihre Wahl

Lösung: Verschiedene Ansätze denkbar. Meine bevorzugte Variante: [→ **ZP:Sheet:8**]

1. **MVP:** der leere Konverter mit hart-gecodeter initialisierter ZPR, Funktions-/Methodenhüllen und passendem *main*-Bereich
2. **VP-0.1:** MVP + Outputadapter, der hart-gecodeten initialisierter ZPR in das **Input**format 'konvertiert'.
3. **VP-0.2:** VP-0.1 + Inputadapter, der die externen Testdaten in die ZPR überführt. (Durchstichskonverter: Inputformat -> Inputformat)
4. **VP-0.3:** VP-0.2 + Outputadapter, der hart-gecodeter initialisierter ZPR in das **Output**format 'konvertiert'
5. **VP-1.0:** VP-0.x + User-Handlingslayer

Begründung:

- Ich schreibe immer zuerst ein leeres, aber strukturell möglichst komplettes Programm, damit der Rest nur 'Erweiterungsarbeit' ist.

- Ich schreibe immer zuerst den Konverter 'Inputformat' nach 'Inputformat'. Das reduziert die Komplexität!

Zwischenübung zum Konverterbau:

ÜBUNG LF11C:Datenanalyse:03

- ☐ Laden Sie sich die Datei `dsp.beta.xyz` herunter.
 - ☐ Durchlaufen Sie mit diesen Daten alle Schritte der syntaktischen Aufbereitung:
 - ☐ Stellen Sie eine Hypothese darüber auf, worum es bei diesen Daten geht.
 - ☐ Finden Sie die Ordnung in den Dateien. (SYDA.A, SYDA.I)
 - ☐ Modifizieren Sie die Daten in der Datei syntaktisch so, dass sie diese Ordnung widerspiegeln, ohne die Aussage zu ändern. (SYDA.F)
 - ☐ Berichtigen Sie die Daten wo nötig syntaktisch. (SYDA.R)
 - ☐ Berichtigen Sie die Daten wo unmittelbar erkennbar und möglich auch schon jetzt semantisch. (SEDA.K)
 - ☐ Programmieren Sie in Python einen Konverter, der diese Daten im INI-Format ausgibt: (SYDA.K)
 - ☐ Gehen Sie dabei gemäß MVP-Strategie vor.
 - ☐ Nutzen Sie so wenig wie möglich externe Bibliotheken. Hinweis:
 - `<protico.lf.cx/cx.datafiles-*>` beschreibt Datenaustauschformate.
-

Lösung:

- (SYDA.A, - SYDA.R + SEDA.K): Datenkorrektur [**→ ZP:Sheet:9**]
- Weltausschnitt der Daten: Tageslängen [**→ ZP:Sheet:10**]
- Konverter MVP: [**→ ZP:Sheet:11**]
 - Die Funktionen sind insofern leer, als sie nur sagen, was sie tun, ohne es tatsächlich zu tun.
- Konverter VP-0.1: [**→ ZP:Sheet:12**]
 - Die hinzugefügte Schreibadapter
 - * iteriert über die Liste von Dictionaries in den hardgecodete Testdaten (L21)
 - * nutzt f-Strings, um einen Datensatz neu zusammenzusetzen und zu schreiben. (L22)
- Konverter VP-0.2/3: [**→ ZP:Sheet:13**]: importierbares Modul mit

- A) einem Leseadapter (L5ff)
 - * liest die CSV zeilenweise ein (L10),
 - * entfernt überflüssige Whitespaces vor und nach dem Datensatz,
 - * spaltet die Zeile anhand der Kommata,
 - * fügt ein entsprechendes Dictionary in die Liste der Datensätze hinzu,
 - B) einem weiteren Schreibadapter (analog zu VP-0.1). (L26ff)
- Konverter VP-1.0: [→ **ZP:Sheet:14**]
 - importiert Adapter-Modul
 - verwendet Kommandozeilenparameter um Zielformate etc. auswählbar zu machen
 - lässt die Daten einlesen (L25)
 - lässt die Daten gemäß Vorgaben schreiben (L29ff)

ÜBUNG LF11C:Datenanalyse:04

Arbeiten Sie diese Aufgabe als Team ab. Organisieren Sie sich gemeinsam so, dass Sie selbst als Teil eines Subteams der Lösung zuarbeiten können. Die Aufgabe ist gelöst, wenn die Klasse gemeinsam einen funktionsfähigen Konverter programmiert hat. Dazu gehen Sie so vor:

- ☐ Laden Sie sich sicherheitshalber die Datei `dsp.alpha.xyz` noch einmal herunter.
- ☐ Finden Sie die Ordnung in den Dateien. (SYDA.A, SYDA.I)
- ☐ **Einigen Sie sich auf ein Format für die Zwischenrepräsentation.** * Das ist der Kern, den alle kennen müssen, um Aufgaben autonom zuarbeiten zu können.*
- ☐ Organisieren Sie sich so, dass folgende Arbeiten möglichst schnell erledigt werden:
 - ☐ Modifizieren Sie die Daten in der Datei syntaktisch so, dass sie diese Ordnung widerspiegeln, ohne die Aussage zu ändern. (SYDA.F)
 - ☐ Berichtigen Sie die Daten wo nötig syntaktisch. (SYDA.R)
 - ☐ Berichtigen Sie die Daten wo unmittelbar erkennbar und möglich auch schon jetzt semantisch. (SEDA.K)
 - ☐ Spalten Sie 10% der bereinigten Daten als Testdaten ab
 - ☐ Programmieren Sie in Python anhand der MVP-Strategie einen Konverter, der diese Daten im JSON-Format ausgibt: (SYDA.K)
 - ☐ Nutzen Sie so wenig wie möglich externe Bibliotheken.
- ☐ Verfeinern Sie nebenbei fortlaufend Ihre Hypothese darüber auf, worum es bei diesen Daten geht.

Lösung: Blutdruckmessungen [**→ ZP:Sheet:15**]

Oder wie Mustafa (12ip/iv23) spontan sagte: “Ahh, der Herr Reincke hat Bluthochdruck”.